Live Coding the Lineage

Phylogenetic Visualisations and Interactive Interfaces for Evolutionary Sound Discovery

Björn Þór Jónsson
RITMO, Dept. of Informatics
University of Oslo
bthj@uio.no

Çağrı Erdem
Dept. of Informatics,
University of Oslo
cagrie@uio.no

Stefano Fasciani
Dept. of Musicology,
University of Oslo
stefanof@uio.no

Kyrre Glette
RITMO, Dept. of Informatics
University of Oslo
kyrrehg@uio.no

ABSTRACT

This paper presents a web-based system for exploring and performing with sounds discovered through Quality Diversity evolutionary algorithms. Our system renders interactive phylogenetic trees of evolutionary sound discovery processes, transforming genealogical relationships into a performable interface—the "Harpsitree"—using trajectory-based interaction and automatic code generation. The core concept we explore is "algorithmic sketching," where trajectory-based discovery through evolutionary structures progressively formalises into executable code, bridging visual exploration and textual live coding paradigms. This work contributes a novel performance interface as part of a larger research project investigating evolutionary computation for sound discovery, demonstrating how phylogenetic structures can reveal compositional affordances through appropriate interaction design.

Keywords

evolutionary computation, web audio, live coding

1. INTRODUCTION

The application of evolutionary computation to sound discovery represents a paradigm shift from optimisation-focused approaches towards open-ended exploration of sonic possibility spaces. Quality Diversity (QD) algorithms [3], particularly MAP-Elites [20], enable the discovery of diverse collections of high-performing solutions by maintaining archives organised according to behavioural characteristics. When applied to sound synthesis through evolving Compositional Pattern Producing Networks (CPPNs) [30] coupled with Digital Signal Processing (DSP) graphs [15], these algorithms can illuminate vast regions of sonic possi-

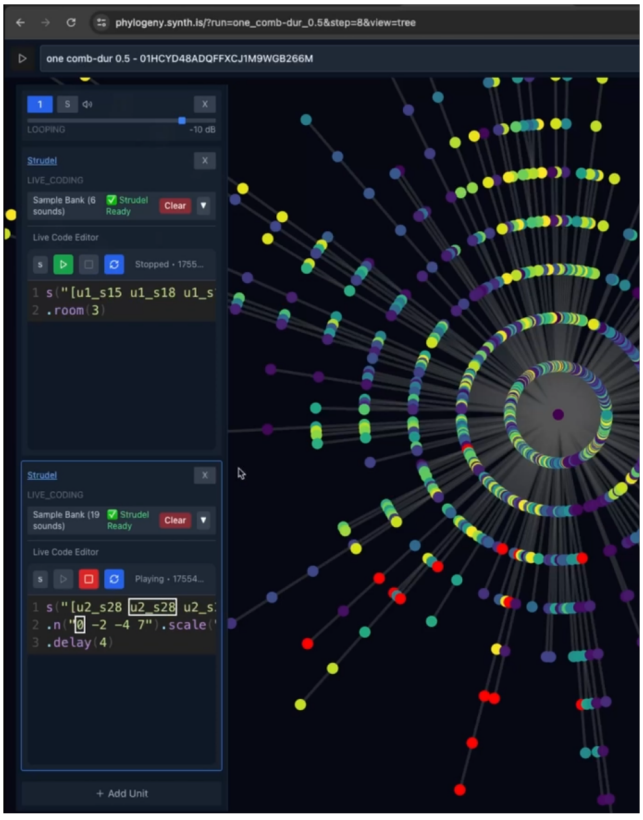


Figure 1: Interactive visualisations of phylogenetic trees evolved during evolutionary simulations. Different types of sequencing units are available and the figure depicts two live coding units based on Strudel: with those units active, interaction with sound-nodes results in live coding snippets to be auto-generated, while manual modifications—such as the addition of a delay and node scales in the figure—are preserved during automatic code updates with visual interaction.

Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution**: owner/author(s).

Web Audio Conference WAC-2025, November 19–21, 2025, Paris, France.

© 2025 Copyright held by the owner/author(s).

bility that might inspire transformational creative processes [11, 10, 12].

However, a fundamental challenge emerges: how can we make these algorithmically discovered sound corpuses accessible and performable whilst preserving the evolutionary relationships that led to their creation? Traditional corpus-based exploration tools like AudioStellar [6] and CataRT [29] arrange sounds according to extracted features in low-dimensional spaces, but these representations do not capture the genealogical paths and relationships inherent in evolutionary processes. Our work addresses this gap through an interactive phylogenetic tree interface that renders complete evolutionary histories as visible, explorable, and performable structures. One state of this interface are shown in Figure 1.

The system enables multiple forms of interaction with evolutionary structures. A part of our approach is the automatic generation of executable code from phylogenetic navigation. By hovering or clicking nodes, users produce code snippets that capture individual sounds along with their evolutionary relationships, enabling integration with live coding practices. This "algorithmic sketching" mechanism creates a concrete bridge between visual exploration and textual live coding, suggesting a new paradigm where trajectory-based discovery progressively formalises into code. Through implementation of trajectory-based sequencing units and integration with the Web Audio API, we explore the implications of temporal displacement in algorithmic creativity—where evolutionary computation creates possibilities during one phase that performers explore and activate during another. Development of this interface revealed how evolutionary structures contain inherent compositional affordances: our explorations resulted in the documentation of over a hundred compelling temporal organisations emerging from lineage navigation, suggesting rich possibilities for phylogenetic performance interfaces.

This paper contributes a novel interface for performing with evolutionary sound discoveries, demonstrating how phylogenetic visualisation transforms algorithmic exploration into musical expression. The "Harpsitree" system bridges exploratory data analysis and live performance through trajectory-based interaction and automatic code generation. We present this work as part of ongoing investigations into how evolutionary computation might extend creative practices in computer music.

2. BACKGROUND AND RELATED WORK

Evolutionary algorithms have been applied to sound synthesis for a variety of purposes and from multiple perspectives, including parameter optimisation for existing synthesisers [8], exploration of vast parameter spaces through interactive evolution [5], and evolution of synthesis architectures themselves [18]. Our approach builds upon previous work using CPPNs coupled with DSP graphs [15], where networks generate patterns serving as audio and control signals.

QD algorithms extend traditional evolutionary approaches by maintaining diverse archives of solutions organised by behavioural characteristics [3]. MAP-Elites, our chosen algorithm, discretises the behaviour space into a grid where each cell maintains the highest-performing solution exhibiting those characteristics [20]. This approach promotes exploration of diverse sonic territories rather than convergence on singular optima.

Live coding traditionally involves writing and modifying code during performance, with the code serving as both composition and score [4]. Several systems already explore multiple representations of algorithmic state: Strudel provides textual patterns with visual timelines [25], Threnoscope offers multiple simultaneous views [19], and Gibber combines code with visual feedback [22]. Recent work has further explored agent-based approaches where autonomous entities navigate sonic spaces [24]. Our system offers a parallel exploration through trajectory-based interaction, which aligns with work in spacetime trajectory control for rhythmic interfaces [23] and visualization techniques for temporal data [7]. Unlike LLM-based code generation that optimizes toward textual targets, our approach directly maps these exploratory trajectories into executable code.

The conceptual framework for our approach resonates with Parncutt's research on pulse salience [21], which offers relevant context on human temporal processing windows that complement our trajectory sequencing approach. Recent research on biological rhythms [17] and movement visualization [31] provides additional grounding for interaction interfaces that leverage natural temporal patterns for navigating complex evolutionary structures.

3. SYSTEM ARCHITECTURE

Web Audio API Across Computing Environments .

The choice to build upon the Web Audio API for sound synthesis emerged from a commitment to accessibility—enabling evolutionary sound discovery directly in web browsers without specialised software installation. However, this decision introduced architectural and technical challenges when scaling our Quality Diversity search to additional execution contexts, ranging from laptops to High Performance Computing (HPC) environments.

The Web Audio API's `setValueCurveAtTime()` method proved particularly crucial for our approach. This method allows CPPN networks to generate arbitrary-length control signals that modulate DSP graph parameters over time, creating complex timbral evolution. Rather than using CPPNs merely as sources of single-cycle waveforms, we employ them to generate both audio-rate signals and control curves that evolve throughout a sound's duration. Each sound is rendered with fixed temporal evolution, but the phylogenetic tree provides thousands of variants to explore, where performers navigate evolutionary diversity rather than synthesis parameters. This temporal richness required significant computational cost during the discovery phase, as each unique frequency requested by the DSP graph required separate CPPN activation at every audio sample during evolutionary search.

A breakthrough in addressing this computational challenge came through our GPU acceleration implmentation, which leverages GPU.js [28] by transforming CPPN networks into shader programs executable on graphics hardware. This approach, functional in both browser and Node.js environments, achieved dramatic performance improvements.

When migrating from browser-based Interactive Evolutionary Computation to distributed HPC systems, we faced a critical decision: abandon the Web Audio API for more portable synthesis libraries, or find ways to maintain API consistency across environments. The discovery of the Node

Web Audio API project[1]—a Rust implementation of the Web Audio specification with Node.js bindings [26]—proved instrumental. This library enabled us to preserve our entire synthesis codebase whilst executing on server clusters, maintaining API compatibility, including the essential `setValueCurveAtTime()` functionality.

From CLI to Interactive Web Interface.

The development journey from command-line interfaces to rich visualisation reveals important insights about exploring evolutionary results. Initial implementations provided basic audio file rendering and automated playback through discovered populations, but manually sampling vast sonic discoveries offered only narrow windows into the evolutionary process. This limitation motivated the development of a six-dimensional exploration interface, enabling navigation through experiments, behavioural classes, generations, durations, pitches, and velocities[2]. Yet even this improved accessibility lacked compositional affordances—arranging slider settings to audition individual sounds failed to capture the richness of evolutionary relationships.

Recognising that phylogenetic relationships themselves could serve as a compositional structure, the web-based interface presented here renders evolutionary histories as interactive trees alongside heatmap representations of the behaviour space, transforming genealogical data into a performable instrument. This approach preserves not just the sounds but the evolutionary narratives that produced them.

Optimisation and Interactivity.

Initial `D3.js/SVG` implementations demonstrated acceptable performance for trees under 1,000 nodes but exhibited significant lag on mobile devices and with larger datasets. This motivated a fundamental architectural revision employing Canvas-based GPU-accelerated rendering with quadtree spatial indexing for logarithmic-time hit detection.

These improvements enable smooth 60 fps interaction with trees containing 10,000+ nodes whilst maintaining visual aesthetics and touch responsiveness across devices.

For audio synthesis and rendering throughout the evolutionary system—both during automated simulations and interactive playback—the `virtual-audio-graph` library[3] provides a declarative abstraction over the Web Audio API. This library enables the CPPN-DSP graph combinations to be represented declaratively, maintaining consistency across the entire pipeline from evolutionary discovery to user interaction. The declarative approach proved useful for managing complex audio graphs that evolve dynamically, where CPPN outputs connect to various DSP nodes at runtime.

In the phylogenetic tree viewer interface specifically, the Elementary library[4] handles the heavy lifting of playback and sequencing capabilities. Elementary's declarative approach with automatic reconciliation proves particularly valuable for responsive multi-voice performance. Where traditional Web Audio programming requires manual creation and connection of audio nodes, Elementary enables functional composition that maintains near-constant latency even with complex polyphonic arrangements. This dual-library approach leverages each tool's strengths: `virtual-audio-graph` for consistent genome representation across the entire system, and Elementary for efficient real-time interaction in the web interface.

Code Generation Architecture.

The automatic code generation system bridges gestural and textual paradigms through semantic translation of user interactions. Hovering or clicking nodes generates basic Strudel snippets that play sounds in sequence, with additional hovered nodes added to a FIFO (first in, first out) queue (default: 4 sounds, 1 repetition each, both controllable via GUI sliders). Users can then manually modify the auto-generated code—adding reverb, filters, delay, or other effects—while further phylogenetic interactions continue generating new code sections without overwriting manual additions. This co-existence of automated generation and manual live coding enables fluid transitions between exploratory gesture and deliberate textual modification during performance. During performance, artists alternate between visual exploration and code editing, creating evolving algorithmic compositions.

This architecture enables direct translation from phylogenetic navigation to executable code, where node interactions generate code snippets that preserve both individual sounds and their evolutionary context. The generated code preserves not only the selected sounds but also their evolutionary context, facilitating textual manipulations that uphold musical coherence anchored in the underlying structure.

4. INTERFACE DESIGN

The Harpsitree: Phylogenetic Performance Interface.

The visual arrangement of phylogenetic branches evokes harp strings, inspiring the "Harpsitree" metaphor. This extends beyond visual similarity—evolutionary relationships provide inherent musical coherence analogous to harmonic relationships between harp strings. Traversing lineages produces sonic narratives reflecting gradual mutation, whilst branch-jumping introduces controlled discontinuity. The capability to "strum" different nodes with immediate, responsive feedback transforms the static phylogenetic structure into a dynamic musical instrument.

User interactions create 2D trajectories through phylogenetic space via mouse or touch input—movement along branches traverses generational time while movement across branches navigates between different evolutionary lineages. These trajectories function as temporal sequences: the timing between mouse movements during navigation determines the intervals between node playbacks during sequence repetition.

Three primary interaction modalities enable various performance approaches. Trajectory units allow free-form exploratory paths through phylogenetic space, which can be considered in relation to spacetime trajectory research for interaction design [23], with dragging interactions triggering sequences through evolutionary neighbours and enabling "strumming" across branches, with the option to record these trajectories for subsequent playback. Sequencing units, inspired by AudioStellar [6], provide explicit arrangement with quantised timing, where users click nodes to build sequences with per-sound controls for duration, pitch, and amplitude.

[1] `https://github.com/ircam-ismm/node-web-audio-api`
[2] `https://youtu.be/crYc1_KVHL4`
[3] `https://github.com/benji6/virtual-audio-graph`
[4] `https://www.elementary.audio`

Looping units enable hover-triggered looped playback of individual sounds.

Genealogical relationships offer unique compositional possibilities aligned with natural rhythmic perception [21]: traversing along branches reveals incremental mutations, moving across the same generational ring exposes parallel evolutionary paths and jumping between distant branches creates evolutionary contrast. These structural properties enable musical trajectories impossible with traditional feature-based arrangements, where proximity reflects perceptual similarity rather than generative history.

Algorithmic Sketching: An Approach to Interactive Code Creation.

Our system explores extensions to live coding practice by enabling algorithmic thinking through trajectory-based interaction with visual representations of computational processes. We propose that live coding fundamentally requires three elements: algorithmic specification (users must define or modify algorithmic behaviours), real-time feedback (changes immediately affect output), and performative transparency (the process remains observable).

This aligns with Blackwell et al.'s Live Algorithms framework [2], which decomposes interactive music systems into analysis, synthesis, and patterning/reasoning components. Our phylogenetic interface manifests these components through: analysis of evolutionary relationships and lineage structures, synthesis via real-time Web Audio API rendering, and patterning/reasoning embedded within the phylogenetic structure itself—where evolutionary history provides the algorithmic logic that traditional live coding would express textually.

The system satisfies these criteria through trajectory-defined algorithmic sequences, immediate audio response, and visual representation of both structure and process. More significantly, the automatic code generation feature creates what we term an "algorithmic sketching environment"—a space where compositional ideas emerge through exploration and are progressively formalised into code.

This bi-directional workflow—from exploratory trajectories to code and back—echoes Paul Klee's concept of "taking a line for a walk" where linear movement acquires musical meaning [1], suggesting that future live coding environments might benefit from multiple simultaneous representations of algorithmic logic. We see it as expanding the spectrum of practices, where the boundaries between coding and not-coding become less binary and more fluid.

From Analysis Tool to Performance Instrument.

Throughout development and testing, the first author documented over a hundred audiovisual recordings exploring the interface's dual nature[5]. As an analytical tool, the phylogenetic visualisation reveals evolutionary dynamics through tree morphology, tracks fitness progression via colour-coded nodes, identifies convergent evolution through spatial clustering, and enables examination of parent-child genetic relationships. Researchers can observe mutation rates, diversity patterns, and breakthrough moments in the evolutionary process.

These analytical features naturally transform into performance affordances: fitness progressions become dynamic arcs, branch density indicates timbral stability or variation, and parent-child relationships enable sonic morphing. This transition from analysis to performance wasn't designed but discovered—patterns in evolutionary data suggested their own musical interpretations. Lineages contain natural crescendos, branch points create structural markers, and evolutionary "dead ends" provide unique colours.

This duality resonates with Rowe's [27] observations about interactive music systems, where the aesthetics emerge from the interplay between systematic processes and performative exploration. The phylogenetic structure provides both a window into algorithmic behaviour and a substrate for musical expression—demonstrating that analytical and creative modalities need not be mutually exclusive in computational music systems.

5. CONCLUSION

This work demonstrates how phylogenetic visualisation transforms algorithmic sound discoveries into performable instruments through trajectory-based navigation and automatic code generation. Our "algorithmic sketching" approach bridges exploratory visualisation and textual live coding, suggesting approaches where performers can fluidly move between interaction paradigms rather than choosing one over another.

The technical commitment to Web Audio API consistency—enabled by Node Web Audio API [26] and GPU.js [28]—validates web technologies for computationally intensive audio applications. This architectural foundation enables natural extensions: real-time evolutionary guidance where user interactions shape ongoing algorithmic processes, collaborative performances across distributed phylogenetic spaces, and deeper integration with existing live coding languages where evolutionary structures become control sources or visual scores.

The phylogenetic interface demonstrates how appropriate interaction design can reveal musical possibilities within evolutionary data structures. By transforming analytical tools into performance instruments, this work shows how algorithmic performance can extend beyond traditional text-based paradigms.

The system is available at `https://phylogeny.synth.is` with source code [9] and sound archives [16, 13, 14] published for community exploration. We invite musicians and researchers to discover the compositional possibilities within these algorithmically evolved sound lineages.

6. ACKNOWLEDGMENTS

This work was supported by the Research Council of Norway through its Centres of Excellence scheme, project number 262762 - RITMO Centre for Interdisciplinary Studies in Rhythm, Time and Motion at the University of Oslo. Computational resources were provided by Sigma2 - the National Infrastructure for High Performance Computing and Data Storage in Norway. We thank the developers of AudioStellar for interface inspiration, and the maintainers of the node-web-audio-api and GPU.js projects for their essential contributions to web audio infrastructure.

[5] `https://video.synth.is`

7. REFERENCES

[1] P. Aichele. Paul Klee's "Rhythmisches": A Recapitulation of the Bauhaus Years. *Zeitschrift für Kunstgeschichte*, 57:75–89, 1994.

[2] T. Blackwell, O. Bown, and M. Young. Live Algorithms: Towards Autonomous Computer Improvisers. In J. McCormack and M. d'Inverno, editors, *Computers and Creativity*, pages 147–174. Springer Berlin Heidelberg, Berlin, Heidelberg, 2012.

[3] K. Chatzilygeroudis, A. Cully, V. Vassiliades, and J.-B. Mouret. Quality-Diversity Optimization: A Novel Branch of Stochastic Optimization. In P. M. Pardalos, V. Rasskazova, and M. N. Vrahatis, editors, *Black Box Optimization, Machine Learning, and No-Free Lunch Theorems*, pages 109–135. Springer International Publishing, Cham, 2021.

[4] N. Collins, A. McLean, J. Rohrhuber, and A. Ward. Live coding in laptop performance. *Org. Sound*, 8(3):321–330, Dec. 2003. Place: USA Publisher: Cambridge University Press.

[5] P. Dahlstedt. Creating and Exploring Huge Parameter Spaces: Interactive Evolution as a Tool for Sound Generation. In *Proceedings of the International Computer Music Conference*, Havana, Cuba, Sept. 2001.

[6] L. Garber, T. Ciccola, and J. Amusategui. AudioStellar, an open source corpus-based musical instrument for latent sound structure discovery and sonic experimentation. In *Proceedings of the International Computer Music Conference*, pages 62–67, 2021.

[7] M. Hlawatsch, P. Leube, W. Nowak, and D. Weiskopf. Flow Radar Glyphs—Static Visualization of Unsteady Flow with Uncertainty. *IEEE Transactions on Visualization and Computer Graphics*, 17(12):1949–1958, 2011.

[8] A. Horner, J. Beauchamp, and L. Haken. Machine Tongues XVI: Genetic Algorithms and Their Application to FM Matching Synthesis. *Computer Music Journal*, 17(4):17, 1993.

[9] B. T. Jónsson. synth-is/phylogenetic-sequencer: 2025-01-15, Jan. 2025.

[10] B. T. Jónsson, C. Erdem, S. Fasciani, and K. Glette. Quality-Diversity Search in Sound Generation: Investigating Innovation Engines for Audio Exploration. *in review*, 2024.

[11] B. T. Jónsson, C. Erdem, S. Fasciani, and K. Glette. Towards Sound Innovation Engines Using Pattern-Producing Networks and Audio Graphs. In *To appear in: Artificial Intelligence in Music, Sound, Art and Design: 9th International Conference, EvoMUSART 2024*, Lecture Notes in Computer Science. Springer International Publishing, 2024.

[12] B. T. Jónsson, C. Erdem, S. Fasciani, and K. Glette. Exploring Definitions of Quality and Diversity in Sonic Measurement Spaces. *in review*, 2025.

[13] B. T. Jónsson, C. Erdem, S. Fasciani, K. Glette, and University Of Oslo. Extended Data for: Quality-Diversity Search in Sound Generation: Investigating Innovation Engines for Audio Exploration, 2025.

[14] B. T. Jónsson, C. Erdem, S. Fasciani, K. Glette, and University Of Oslo. Supporting data for: Exploring Definitions of Quality and Diversity in Sonic Measurement Spaces, 2025.

[15] B. T. Jónsson, C. Erdem, and K. Glette. A System for Sonic Explorations with Evolutionary Algorithms. *Journal of the Audio Engineering Society*, 72(4), 2024.

[16] B. T. Jónsson, K. Glette, C. Erdem, S. Fasciani, and University Of Oslo. Supporting Data for: Towards Sound Innovation Engines Using Pattern-Producing Networks and Audio Graphs, 2024.

[17] L. Laffi and others. The Rhythm of Horse Gaits. *Annals of the New York Academy of Sciences*, 1543(1):86–93, 2025.

[18] M. Macret and P. Pasquier. Automatic design of sound synthesizers as pure data patches using coevolutionary mixed-typed cartesian genetic programming. In *Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation*, pages 309–316, Vancouver BC Canada, July 2014. ACM.

[19] T. Magnusson. The threnoscope: A musical work for live coding performance. 2013.

[20] J.-B. Mouret and J. Clune. Illuminating search spaces by mapping elites, Apr. 2015. arXiv:1504.04909 [cs, q-bio].

[21] R. Parncutt. A Perceptual Model of Pulse Salience and Metrical Accent in Musical Rhythms. *Music Perception*, 11(4):409–464, 1994.

[22] C. Roberts, I. Hattwick, E. Sheffield, and G. Smith. Rethinking networked collaboration in the live coding environment Gibber. In *Proceedings of the International Conference on New Interfaces for Musical Expression*, 2022.

[23] D. Rocchesso, A. Bellino, G. Ferrara, and A. Perez. Spacetime Trajectories as Overlapping Rhythms. *International Journal of Human-Computer Studies*, 192:103358, 2024.

[24] G. Roma. Agent-Based Music Live Coding: Sonic adventures in 2D. *Organised Sound*, 28(2):231–240, Aug. 2023. Publisher: Cambridge University Press.

[25] F. Roos and A. McLean. Strudel: Live Coding Patterns on the Web. In *Proceedings of the 7th International Conference on Live Coding*, 2023.

[26] O. Rottier and B. Matuszewski. A Rust Implementation of the Web Audio API. In *Proceedings of the International Web Audio Conference*, July 2022.

[27] R. Rowe. The aesthetics of interactive music systems. *Contemporary music review*, 18(3):83–87, 1999. Publisher: Taylor & Francis.

[28] F. Sapuan, M. Saw, and E. Cheah. General-Purpose Computation on GPUs in the Browser Using gpu.js. *Computing in Science & Engineering*, 20(1):33–42, Jan. 2018.

[29] D. Schwarz. Concatenative sound synthesis: The early years. *Journal of New Music Research*, 35(1):3–22, 2006. Publisher: Taylor & Francis.

[30] K. O. Stanley. Compositional pattern producing networks: A novel abstraction of development. *Genetic Programming and Evolvable Machines*, 8(2):131–162, June 2007.

[31] J. Zhao, P. Forer, and A. S. Harvey. Activities, Ringmaps and Geovisualization of Large Human

Movement Fields. *Information Visualization*, 7(3-4):198–209, 2008.